ZENIT

CÓMO FUNCIONA EL CÓDIGO

Guía para entender ZENIT parte por parte: qué archivo hace qué, cómo se conectan y el código real de cada funcionalidad, explicado en simple.

00 Seguridad de config.ini
01 Mapa general del proyecto
02 La base de datos
03 Registrarse
04 El código por mail
05 Iniciar sesión
06 Mapa, país y ciudad
07 El botón JUGAR paso a paso
08 Las fases de una partida
09 La ruleta
10 La categoría Z
11 Preguntas y reloj
12 Vidas y fin de ronda
13 Ronda de letra y premio
14 Puntos (elo) y estadísticas
15 El perfil y el nivel
16 El código QR
17 Reportar preguntas
18 Fin de la partida
19 Qué archivo hace qué

Programación Web 2 · UNLaM · PHP + MySQL · MVC propio · zenit.free.nf

00 · SEGURIDAD

¿ESTÁ SEGURO CONFIG.INI?

Sí. Acá está por qué, en tres capas.

1. **No está en GitHub.** El archivo `.gitignore` le dice a Git que nunca lo suba. Por eso en el repo solo está `config.example.ini`, un modelo sin contraseñas.

2. **El deploy no lo pisa.** El archivo del deploy (`.github/workflows/deploy.yml`) tiene `config/config.ini` en su lista de exclusiones: GitHub nunca lo toca en InfinityFree.

3. **Nadie lo puede abrir desde el navegador.** El `.htaccess` bloquea la carpeta `config` y cualquier archivo `.ini`. Lo probé en zenit.free.nf: `/config/config.ini` responde **403 Prohibido**.

.gitignore — línea 1

```
# Configuración local con contraseñas (usar config/config.example.ini como modelo)
config/config.ini
```

.htaccess — línea 10

```
# Archivos sensibles: nunca se descargan desde el navegador.
<FilesMatch "\.(ini|log|sql|md|yml)$">
    Require all denied
</FilesMatch>

# Registro interno del deploy automático (lista de archivos subidos): no es público.
<Files ".ftp-deploy-sync-state.json">
    Require all denied
</Files>

<IfModule mod_rewrite.c>
    RewriteEngine On

    # Sin RewriteBase: así funciona igual en http://localhost/ZENIT-TPW2/
    # y en la raíz de un hosting, sin tocar nada.

    # Código interno: solo lo carga index.php. Acceso directo → 403.
    RewriteRule ^(app|helper|vendor|config|log|herramientas)(/|$) - [F,L]

    # Archivos estáticos (css, js, fotos subidas y la carpeta media): se sirven tal cual.
    RewriteRule ^(public|media)/ - [L]
```

Lo único que podría exponer tus claves sería que alguien entre a tu cuenta de InfinityFree o de GitHub. Por eso conviene que tengan contraseñas distintas y la verificación en 2 pasos activada.

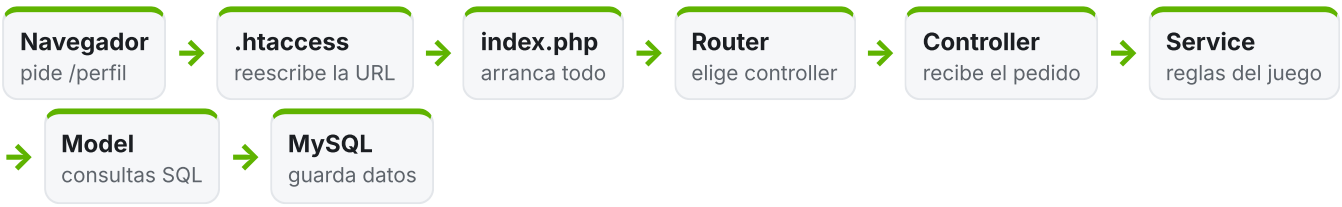
También agregué al `.htaccess` un bloqueo para `.ftp-deploy-sync-state.json` (el registro que deja el deploy con la lista de archivos). No tiene claves, pero no hace falta que sea público. Se aplica en tu próximo `git push`.

01 · ARQUITECTURA

MAPA GENERAL DEL PROYECTO

Cómo viaja un pedido desde que tocás algo hasta que ves la respuesta.

ZENIT usa el patrón **MVC** (Modelo · Vista · Controlador) sin frameworks. Cada parte tiene una sola tarea:



CAPA	CARPETA	QUÉ HACE	EJEMPLO
Controller	`app/controller`	Recibe el pedido, llama a los servicios y elige qué vista mostrar.	JuegoController
Service	`app/service`	La lógica: reglas, cálculos, validaciones. No sabe de HTML.	JuegoService, NivelService
Model	`app/model`	Solo habla con la base de datos (SELECT, INSERT, UPDATE).	UsuarioModel
View	`app/view`	Plantillas Mustache: el HTML con huecos {{así}}.	perfilView.mustache
Helper	`helper`	Herramientas generales: sesión, base, router, mails.	Session, MyDatabase
JS	`public/js`	Lo que pasa en el navegador sin recargar: ruleta, mapa, QR.	juego.js

1. El .htaccess convierte la URL en parámetros

`/perfil/ver` se transforma en `index.php?controller=perfil&method=ver`. Así todas las páginas entran por un único archivo.

.htaccess — línea 21

```
RewriteEngine On

# Sin RewriteBase: así funciona igual en http://localhost/ZENIT-TPW2/
# y en la raíz de un hosting, sin tocar nada.

# Código interno: solo lo carga index.php. Acceso directo → 403.
RewriteRule ^(app|helper|vendor|config|log|herramientas)(/|$) - [F,L]

# Archivos estáticos (css, js, fotos subidas y la carpeta media): se sirven tal cual.
RewriteRule ^(public|media)/ - [L]

# Si no es un archivo ni carpeta real, lo resuelve el router:
#   /perfil/ver?u=demo  →  index.php?controller=perfil&method=ver&u=demo
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^([^/]+)/([^/]+)/?$ index.php?controller=$1&method=$2 [L,QSA]

RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
```

```
    RewriteRule ^([^/]+)/?$ index.php?controller=$1 [L,QSA]
</IfModule>
```

2. index.php arranca la sesión y llama al Router

index.php — línea 13

```php
session_set_cookie_params([
    'httponly' => true,   // JS no puede leer la cookie de sesión
    'samesite' => 'Lax',  // protege los POST contra CSRF básico
]);
session_start();

require_once __DIR__ . '/app/Configuration.php';

$configuration = new Configuration();
$router = $configuration->getRouter();

$router->dispatch(
    Request::get('controller', 'home'),
    Request::get('method', 'show')
);
```

3. El Router busca el controller en Configuration

Si la URL dice `juego`, busca el método `getJuegoController()`. Si no existe, va al lobby. Solo se pueden llamar métodos **públicos**, así nadie puede ejecutar funciones internas desde la URL.

helper/Router.php — línea 22

```php
public function dispatch(string $nombreController, string $metodo): void
{
    $controller = $this->obtenerController($nombreController);
    $metodo = $this->esMetodoPublico($controller, $metodo) ? $metodo : $this->metodoPorDefecto;
    $controller->{$metodo}();
}
```

helper/Router.php — línea 29

```php
private function obtenerController(string $nombre): object
{
    $fabrica = 'get' . ucfirst(strtolower($nombre)) . 'Controller';

    if (!method_exists($this->configuration, $fabrica)) {
        $fabrica = 'get' . ucfirst($this->controllerPorDefecto) . 'Controller';
    }
    return $this->configuration->{$fabrica}();
}
```

4. Configuration arma cada objeto (inyección de dependencias)

Es el **único** archivo que sabe qué necesita cada clase. Por ejemplo, el juego necesita la sesión, la ruleta, los modelos de categorías, preguntas, estadísticas y partidas:

app/Configuration.php — línea 94

```php
public function getJuegoController(): JuegoController
{
    return new JuegoController($this->getRender(), $this->getJuegoService(), $this->getCategoriaModel(),
```

```
        new ReporteService(new EstadoJuego(), new ReporteModel($this->getDatabase())));
}
```

app/Configuration.php línea 120

```
private function getJuegoService(): JuegoService
{
    return new JuegoService(new EstadoJuego(), new RuletaService(), $this->getCategoriaModel(),
        new PreguntaModel($this->getDatabase()), $this->getEstadisticaModel(),
        new PartidaModel($this->getDatabase()));
}
```

¿Por qué así? Si mañana cambiás cómo se guardan las partidas, tocás un solo lugar. Esto es la "D" de SOLID: las clases reciben lo que necesitan, no lo crean ellas.

02 · DATOS

LA BASE DE DATOS

Siete tablas. Todo lo que se guarda vive acá (app/sql/zenit.sql).

TABLA	QUÉ GUARDA	SE USA EN
usuario	Cuenta (nombre completo, usuario, email, contraseña cifrada, país, ciudad, coordenadas, foto), código de verificación y los totales: puntos (elo), respondidas, correctas, partidas jugadas y ganadas.	Registro, login, perfil, juego
categoria	Las 8 categorías: nombre, emoji, color.	Ruleta, perfil
pregunta	El enunciado y a qué categoría pertenece (10 por categoría).	Juego
opcion	6 opciones por pregunta; una tiene `es_correcta = 1`.	Juego
usuario_categoria	Cuántas respondió y acertó cada jugador en cada categoría.	Perfil (% por categoría)
partida	Cada partida: resultado, puntos, letras, respondidas, correctas, fechas.	Perfil (historial)
reporte	Preguntas reportadas: cuál, quién, motivo, comentario, estado.	Futuro panel de admin

app/sql/zenit.sql — línea 18

```
CREATE TABLE usuario (
    id                  INT AUTO_INCREMENT PRIMARY KEY,
    nombre_completo     VARCHAR(150) NOT NULL,
    nombre_usuario      VARCHAR(20)  NOT NULL UNIQUE,
    email               VARCHAR(150) NOT NULL UNIQUE,
    password            VARCHAR(255) NOT NULL,
    anio_nacimiento     SMALLINT     NOT NULL,
    sexo                ENUM('masculino','femenino','no_informa') NOT NULL,
    pais                VARCHAR(80)  NOT NULL,
    ciudad              VARCHAR(80)  NOT NULL,
    latitud             DECIMAL(10,7) NULL,
    longitud            DECIMAL(10,7) NULL,
    foto_perfil         VARCHAR(255) NULL,
    rol                 ENUM('jugador','editor','admin') NOT NULL DEFAULT 'jugador',
    activo              TINYINT(1)   NOT NULL DEFAULT 0,
    -- Código de 6 dígitos enviado por mail (registro, cambio de contraseña o baja)
    codigo_verificacion  CHAR(6)     NULL,
    codigo_proposito     ENUM('registro','password','eliminar') NULL,
    codigo_expira        DATETIME    NULL,
    codigo_intentos      TINYINT     NOT NULL DEFAULT 0,
    -- Estadísticas acumuladas (el "elo" del perfil = puntos)
    puntos              INT NOT NULL DEFAULT 0,
    respondidas         INT NOT NULL DEFAULT 0,
    correctas           INT NOT NULL DEFAULT 0,
    partidas_jugadas    INT NOT NULL DEFAULT 0,
    partidas_ganadas    INT NOT NULL DEFAULT 0,
    creado              TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Cómo habla PHP con MySQL: MyDatabase

Todas las consultas son **preparadas**: los datos van aparte, en lugar de los `?`. Así es imposible el "SQL injection" (que alguien escriba código SQL en un formulario).

helper/MyDatabase.php línea 25

```php
/** Devuelve una fila o null. */
public function queryOne(string $sql, array $parametros = []): ?array
{
    $fila = $this->preparar($sql, $parametros)->fetch();
    return $fila === false ? null : $fila;
}
```

helper/MyDatabase.php línea 49

```php
private function preparar(string $sql, array $parametros): PDOStatement
{
    $sentencia = $this->conexion->prepare($sql);
    $sentencia->execute($parametros);
    return $sentencia;
}
```

app/sql/zenit.sql línea 85

```sql
CREATE TABLE partida (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    usuario_id  INT NOT NULL,
    resultado   ENUM('en_curso','victoria','derrota') NOT NULL DEFAULT 'en_curso',
    puntos      INT NOT NULL DEFAULT 0,
    letras      VARCHAR(5) NOT NULL DEFAULT '',
    respondidas INT NOT NULL DEFAULT 0,
    correctas   INT NOT NULL DEFAULT 0,
    creada      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    terminada   DATETIME NULL,
    FOREIGN KEY (usuario_id) REFERENCES usuario(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- ── Reportes de preguntas (se hacen al terminar una ronda) ──
-- Los revisará el editor/administrador en el panel (próxima entrega).
CREATE TABLE reporte (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    pregunta_id INT NOT NULL,
    usuario_id  INT NOT NULL,
    motivo      ENUM('ofensiva','respuesta_incorrecta','confusa','otro') NOT NULL,
    detalle     VARCHAR(500) NULL,
    estado      ENUM('pendiente','revisado','descartado') NOT NULL DEFAULT 'pendiente',
    creado      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (pregunta_id) REFERENCES pregunta(id) ON DELETE CASCADE,
    FOREIGN KEY (usuario_id)  REFERENCES usuario(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

03 · CUENTA

REGISTRARSE

Del formulario a la cuenta creada (inactiva hasta confirmar el código).



1. El usuario completa el formulario de `registroView.mustache` y toca **Crear cuenta**. El navegador manda un POST a `/registro/registrar`.
2. `RegistroController::registrar()` lee los campos con `leerFormulario()`.
3. `ValidadorRegistro` revisa **todo en el servidor**: campos completos, nombre completo, usuario único y con formato válido, email válido y libre, contraseñas iguales y de 8+ caracteres, año, sexo y país de la lista oficial.
4. Si algo falla, vuelve a mostrar el formulario con el error y lo que ya había escrito (nunca la contraseña).
5. Si está todo bien, `UsuarioModel::crear()` inserta la cuenta con `activo = 0` y la contraseña cifrada con `password_hash`.
6. Se genera un código de 6 dígitos, se manda por mail y se lleva al usuario a `/registro/verificar`.

app/controller/RegistroController.php — línea 46

```php
public function registrar(): void
{
    $datos = $this->leerFormulario();

    $error = $this->validador->validar($datos);
    if ($error !== null) {
        $this->mostrarFormularioConError($error, $datos);
        return;
    }

    $id = $this->usuarios->crear($datos);
    $this->enviarCodigo($this->usuarios->buscarPorId($id));
    Redirect::to('registro/verificar');
}
```

app/controller/RegistroController.php — línea 104

```php
private function leerFormulario(): array
{
    $latitud = Request::texto('latitud');
    $longitud = Request::texto('longitud');
    return [
        'nombre_completo' => preg_replace('/\s+/u', ' ', Request::texto('nombre_completo')),
        'nombre_usuario' => Request::texto('nombre_usuario'),
        'email' => strtolower(Request::texto('email')),
        'password' => (string) Request::post('password', ''),
        'confirmar_password' => (string) Request::post('confirmar_password', ''),
        'anio_nacimiento' => Request::texto('anio_nacimiento'),
        'sexo' => Request::texto('sexo'),
        'pais' => Request::texto('pais'),
```

```
        'ciudad' => Request::texto('ciudad'),
        'latitud' => $latitud !== '' ? $latitud : null,
        'longitud' => $longitud !== '' ? $longitud : null,
    ];
}
```

app/service/ValidadorRegistro.php — línea 21

```
/**
 * @param array $d datos del formulario (el país se corrige al nombre oficial de la lista)
 * @return string|null mensaje del primer error, o null si está todo bien.
 */
public function validar(array &$d): ?string
{
    foreach (['nombre_completo', 'nombre_usuario', 'email', 'password', 'anio_nacimiento', 'sexo', 'pais',
'ciudad'] as $campo) {
        if (($d[$campo] ?? '') === '') {
            return 'Completá todos los campos.';
        }
    }

    if ($error = self::validarNombreCompleto($d['nombre_completo'])) {
        return $error;
    }
    if ($error = $this->validarNombreUsuario($d['nombre_usuario'])) {
        return $error;
    }
    if (!filter_var($d['email'], FILTER_VALIDATE_EMAIL)) {
        return 'Ingresá un email válido.';
    }
    if ($this->usuarios->existeEmail($d['email'])) {
        return 'Ese email ya está registrado.';
    }
    if ($error = self::validarPassword($d['password'], $d['confirmar_password'] ?? '')) {
        return $error;
    }

    $anio = (int) $d['anio_nacimiento'];
    if (!ctype_digit((string) $d['anio_nacimiento']) || $anio < 1900 || $anio > (int) date('Y')) {
        return 'Ingresá un año de nacimiento válido.';
    }
    if (!in_array($d['sexo'], self::SEXOS, true)) {
        return 'Elegí una opción de sexo válida.';
    }
    return $this->validarUbicacion($d);
}
```

app/service/ValidadorRegistro.php — línea 69

```
/** Formato y unicidad del usuario (se elige una sola vez, al registrarse). */
public function validarNombreUsuario(string $nombre): ?string
{
    if (!preg_match('/^[A-Za-z0-9_.]{3,20}$/', $nombre)) {
        return 'El usuario debe tener entre 3 y 20 caracteres (letras, números, punto o guion bajo).';
    }
    if ($this->usuarios->existeNombreUsuario($nombre)) {
        return 'Ese nombre de usuario ya está ocupado.';
    }
    return null;
}
```

app/model/UsuarioModel.php — línea 51

```
/** Crea la cuenta INACTIVA (se activa con el código del mail) y devuelve su id. */
public function crear(array $datos): int
```

```
{
    $this->db->execute(
        'INSERT INTO usuario
            (nombre_completo, nombre_usuario, email, password, anio_nacimiento, sexo, pais, ciudad,
latitud, longitud, activo)
         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, 0)',
        [
            $datos['nombre_completo'],
            $datos['nombre_usuario'],
            $datos['email'],
            password_hash($datos['password'], PASSWORD_DEFAULT),
            $datos['anio_nacimiento'],
            $datos['sexo'],
            $datos['pais'],
            $datos['ciudad'],
            $datos['latitud'],
            $datos['longitud'],
        ]
    );
    return $this->db->lastInsertId();
}
```

La contraseña nunca se guarda tal cual. `password_hash` la convierte en un texto irreversible. En el login, `password_verify` compara sin necesidad de "descifrarla".

04 · CUENTA

EL CÓDIGO POR MAIL

Cómo se genera, se manda, se valida y se activa la cuenta.

1. `VerificacionService::enviar()` genera un número al azar seguro con `random_int` (000000 a 999999) y calcula el vencimiento: **15 minutos**.
2. Lo guarda en la tabla `usuario` junto con su **propósito** (registro, password o eliminar) y pone los intentos en 0.
3. `CorreoService` arma el mail HTML con el logo y `SmtpMailer` lo envía por Gmail (SMTP, puerto 587, cifrado TLS).
4. El usuario escribe el código. `RegistroController::confirmar()` llama a `verificar()`, que revisa: intentos (máximo 5), propósito, vencimiento y que coincida.
5. Si es correcto, el código se borra (no se puede reusar), la cuenta pasa a `activo = 1` y se inicia la sesión.

app/service/VerificacionService.php — línea 39

```php
/**
 * Genera un código nuevo, lo guarda y lo manda por mail.
 * @return bool true si el mail salió.
 */
public function enviar(array $usuario, string $proposito): bool
{
    $codigo = str_pad((string) random_int(0, 999999), 6, '0', STR_PAD_LEFT);
    $expira = date('Y-m-d H:i:s', time() + self::VIGENCIA_MINUTOS * 60);
    $this->usuarios->guardarCodigo((int) $usuario['id'], $codigo, $proposito, $expira);

    return $this->correo->codigo($usuario, $codigo, $proposito, self::VIGENCIA_MINUTOS);
}
```

app/service/VerificacionService.php — línea 52

```php
/** Valida el código; si es correcto lo consume (no se puede reutilizar). */
public function verificar(array $usuario, string $codigo, string $proposito): string
{
    if ((int) $usuario['codigo_intentos'] >= self::MAX_INTENTOS) {
        return self::BLOQUEADO;
    }
    if ($usuario['codigo_proposito'] !== $proposito
        || empty($usuario['codigo_expira'])
        || strtotime($usuario['codigo_expira']) < time()) {
        return self::EXPIRADO;
    }
    if (!hash_equals((string) $usuario['codigo_verificacion'], $codigo)) {
        $this->usuarios->sumarIntentoFallido((int) $usuario['id']);
        return self::INCORRECTO;
    }

    $this->usuarios->limpiarCodigo((int) $usuario['id']);
    return self::OK;
}
```

app/model/UsuarioModel.php — línea 74

```php
public function guardarCodigo(int $id, string $codigo, string $proposito, string $expira): void
{
    $this->db->execute(
        'UPDATE usuario
         SET codigo_verificacion = ?, codigo_proposito = ?, codigo_expira = ?, codigo_intentos = 0
         WHERE id = ?',
        [$codigo, $proposito, $expira, $id]
    );
}
```

app/controller/RegistroController.php línea 71

```php
public function confirmar(): void
{
    $usuario = $this->usuarioPendiente();
    $codigo = preg_replace('/\D/', '', Request::texto('codigo'));
    $resultado = $this->verificacion->verificar($usuario, $codigo, VerificacionService::REGISTRO);

    if ($resultado !== VerificacionService::OK) {
        Session::flash('error', VerificacionService::MENSAJES[$resultado]);
        Redirect::to('registro/verificar');
    }

    $this->usuarios->activar((int) $usuario['id']);
    Session::borrar('verificar_id');
    Session::iniciar($usuario);
    Session::flash('ok', '¡Cuenta verificada! Bienvenido a ZENIT, ' . $usuario['nombre_usuario'] . '.');
    Redirect::to(Session::tomarDestino());
}
```

`hash_equals` compara el código en un tiempo constante: así no se puede "adivinar" midiendo cuánto tarda la respuesta. El propósito evita que un código de registro sirva para borrar la cuenta.

05 · CUENTA

INICIAR SESIÓN

Usuario o email + contraseña, y la vuelta automática a donde querías ir.

1. El formulario de `loginView` manda `usuario` y `password` a `/login/login`.
2. `UsuarioModel::buscarParaLogin()` busca una cuenta cuyo **nombre de usuario O email** coincida.
3. `password_verify()` compara la contraseña escrita con la cifrada. Si no coincide, el mensaje es genérico ("usuario o contraseña incorrectos") para no revelar qué usuarios existen.
4. Si la cuenta no está verificada, manda un código nuevo y lo lleva a verificar.
5. `Session::iniciar()` guarda el id del usuario en `$_SESSION` y regenera el id de sesión (protege contra robo de sesión).
6. Redirige a la página que había pedido antes. Ejemplo: tocó JUGAR sin cuenta → login → vuelve al juego.

app/controller/LoginController.php — línea 32

```
public function login(): void
{
    $usuarioOEmail = Request::texto('usuario');
    $password = (string) Request::post('password', '');
    $usuario = $usuarioOEmail !== '' ? $this->usuarios->buscarParaLogin($usuarioOEmail) : null;

    if ($usuario === null || !password_verify($password, $usuario['password'])) {
        $this->render->renderiza('login', [
            'pantalla_auth' => true,
            'error' => 'Usuario o contraseña incorrectos.',
            'usuario' => $usuarioOEmail,
        ]);
        return;
    }

    // Cuenta sin verificar: le mandamos un código nuevo y lo llevamos a validarlo.
    if (!(int) $usuario['activo']) {
        Session::set('verificar_id', (int) $usuario['id']);
        Session::set('ultimo_envio', time());
        $enviado = $this->verificacion->enviar($usuario, VerificacionService::REGISTRO);
        Session::flash(
            $enviado ? 'info' : 'error',
            $enviado ? 'Tu cuenta todavía no está verificada. Te mandamos un código nuevo al mail.'
                     : 'Tu cuenta no está verificada y no pudimos enviar el mail. Probá reenviar el
código.'
        );
        Redirect::to('registro/verificar');
    }

    Session::iniciar($usuario);
    Redirect::to(Session::tomarDestino()); // ej: tocó JUGAR sin cuenta → vuelve al juego
}
```

app/model/UsuarioModel.php — línea 24

```
/** El login acepta nombre de usuario o email. */
public function buscarParaLogin(string $usuarioOEmail): ?array
{
    return $this->db->queryOne(
```

```
        'SELECT * FROM usuario WHERE nombre_usuario = ? OR email = ?',
        [$usuarioOEmail, $usuarioOEmail]
    );
}
```

Cómo se "recuerda" a dónde volver

Cada página privada llama a `Session::requerirLogin()`. Si no hay sesión, guarda la ruta pedida en `volver` y manda al login. Después del login, `tomarDestino()` la lee una sola vez.

helper/Session.php — línea 10

```
public static function iniciar(array $usuario): void
{
    session_regenerate_id(true);
    $_SESSION['usuario_id'] = (int) $usuario['id'];
}
```

helper/Session.php — línea 26

```
/**
 * Corta la ejecución y manda al login si no hay sesión.
 * Recuerda la página pedida para volver ahí después de entrar
 * (ej: tocó JUGAR sin cuenta → login → juego).
 */
public static function requerirLogin(string $mensaje = 'Iniciá sesión o creá una cuenta para continuar.'):
int
{
    $id = self::usuarioId();
    if ($id === null) {
        if (!Request::esPost()) { // un formulario a medias no se repite después del login
            self::set('volver', self::rutaActual());
        }
        self::flash('info', $mensaje);
        Redirect::to('login');
    }
    return $id;
}
```

helper/Session.php — línea 44

```
/** Ruta a la que hay que volver después del login ('' = lobby). Se usa una sola vez. */
public static function tomarDestino(): string
{
    $destino = (string) self::get('volver', '');
    self::borrar('volver');
    return $destino;
}
```

> La cookie de sesión se configura en `index.php` con `httponly` (JavaScript no la puede leer) y `samesite=Lax` (otras webs no pueden enviar formularios en tu nombre).

06 · UBICACIÓN

MAPA, PAÍS Y CIUDAD

Qué servicio externo hace cada cosa y cómo se conectan.

PIEZA	QUÉ ES	PARA QUÉ SE USA
Leaflet	Librería JS de mapas (gratis, se carga desde un CDN).	Dibujar el mapa, el marcador y recibir los clics.
OpenStreetMap	Los "mosaicos" (imágenes) del mapa del mundo.	El fondo del mapa. Se oscurece con un filtro CSS.
Nominatim	Buscador de OpenStreetMap.	Clic en el mapa → qué país y ciudad es (geocodificación inversa).
Photon	Autocompletado de lugares (OpenStreetMap).	Sugerencias de ciudades mientras escribís.
paises.json	Lista propia de países con su código.	Sugerencias de país y validación en el servidor.
flagcdn.com	Imágenes de banderas.	La bandera al lado del país.

Cómo se conecta todo

1. La vista incluye `partials/ubicacion.mustache`: el mapa, los campos País y Ciudad, y dos campos ocultos `latitud` y `longitud`.
2. `partials/scripts-mapa.mustache` carga Leaflet y `public/js/ubicacion.js`.
3. `ubicacion.js` descarga `paises.json` y arranca el mapa en la vista del mundo.
4. Al **tocar el mapa**: pone el marcador, guarda latitud/longitud en los campos ocultos y le pregunta a Nominatim qué lugar es. Completa país y ciudad solos.
5. Al **escribir el país**: filtra la lista local (primero los que empiezan con lo escrito, después los que lo contienen) y muestra 8 con bandera. Al elegir, centra el mapa en ese país.
6. Al **escribir la ciudad**: espera 300 ms (para no consultar en cada tecla), le pregunta a Photon y deja solo ciudades del país elegido. Al elegir, mueve el marcador.
7. Al enviar el formulario, **el servidor vuelve a validar**: el país tiene que estar en `paises.json` y las coordenadas tienen que ser válidas.

app/view/partials/ubicacion.mustache — línea 6

```
<div class="ubicacion" data-ubicacion>
    <div class="campo">
        <span>País y ciudad — tocá en el mapa o escribí</span>
        <div class="mapa" data-mapa></div>
    </div>
    <div class="fila-2">
        <label class="campo autocompletar">
            <span>País</span>
            <input type="text" name="pais" value="{{pais}}" maxlength="80" autocomplete="off" required
data-pais>
            <ul class="sugerencias" role="listbox" hidden></ul>
```

```
        </label>
        <label class="campo autocompletar">
            <span>Ciudad</span>
            <input type="text" name="ciudad" value="{{ciudad}}" maxlength="80" autocomplete="off" required
data-ciudad>
            <ul class="sugerencias" role="listbox" hidden></ul>
        </label>
    </div>
    <input type="hidden" name="latitud" value="{{latitud}}" data-lat>
    <input type="hidden" name="longitud" value="{{longitud}}" data-lon>
    <p class="ayuda" data-estado aria-live="polite"></p>
```

public/js/ubicacion.js — línea 23

```
fetch(`${BASE}/public/data/paises.json`)
    .then((r) => r.json())
    .then((lista) => { PAISES = lista.map((p) => [p.codigo, p.nombre]); })
    .catch(() => { /* sin lista: igual se puede escribir y el servidor valida */ })
    .finally(() => document.querySelectorAll('[data-ubicacion]').forEach(iniciar));
```

public/js/ubicacion.js — línea 43

```
if (window.L && contenedorMapa) {
    // Mapa del mundo entero (mosaicos libres de OpenStreetMap, sin API key)
    mapa = L.map(contenedorMapa, {
        minZoom: 1,
        maxZoom: 18,
        zoomSnap: 0.25,          // permite encuadrar el planeta completo
        worldCopyJump: true,
        maxBounds: [[-85, -540], [85, 540]],
        maxBoundsViscosity: 0.8,
    });
    L.tileLayer('https://tile.openstreetmap.org/{z}/{x}/{y}.png', {
        maxZoom: 18,
        attribution: '&copy; <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a>',
    }).addTo(mapa);
    mapa.fitWorld();

    if (lat.value && lon.value) {
        ponerMarcador(parseFloat(lat.value), parseFloat(lon.value)); // se marca sin perder la vista del
mundo
    }
    mapa.on('click', (e) => {
        ponerMarcador(e.latlng.lat, e.latlng.lng);
        lugarDesdeCoordenadas(e.latlng.lat, e.latlng.lng);
    });
```

public/js/ubicacion.js — línea 83

```
async function lugarDesdeCoordenadas(latitud, longitud) {
    estado.textContent = 'Buscando ciudad…';
    try {
        const url = `${NOMINATIM}/reverse?format=jsonv2&accept-
language=es&zoom=10&lat=${latitud}&lon=${longitud}`;
        const d = (await (await fetch(url)).json()).address || {};
        if (!d.country) {
            estado.textContent = 'Ahí no hay un país. Probá otro punto.';
            return;
        }
        pais.value = nombrePais(d.country_code) || d.country;
        ciudad.value = d.city || d.town || d.village || d.municipality || d.county || d.state || '';
        estado.textContent = `📍 ${ciudad.value ? ciudad.value + ', ' : ''}${pais.value}`;
    } catch (e) {
        estado.textContent = 'No se pudo consultar el mapa. Escribí país y ciudad.';
```

```
    }
}
```

public/js/ubicacion.js línea 111

```
autocompletar(pais, 0, (texto) => {
    const q = normalizar(texto);
    if (!q) return [];
    const empiezan = PAISES.filter((p) => normalizar(p[1]).startsWith(q));
    const contienen = PAISES.filter((p) => !normalizar(p[1]).startsWith(q) &&
normalizar(p[1]).includes(q));
    return empiezan.concat(contienen).slice(0, 8).map((p) => ({ texto: p[1], codigo: p[0], bandera:
bandera(p[0]) }));
}, (item) => {
    pais.value = item.texto;
    ciudad.value = '';
    centrarEnPais(item.texto);
    ciudad.focus();
});
```

public/js/ubicacion.js línea 134

```
autocompletar(ciudad, 300, async (texto) => {
    if (normalizar(texto).length < 2) return [];
    const codigo = codigoPais();
    const centro = mapa ? mapa.getCenter() : null;
    const params = new URLSearchParams({ q: texto, limit: 20, osm_tag: 'place' });
    if (centro) { params.set('lat', centro.lat); params.set('lon', centro.lng); }

    const datos = await (await fetch(`${PHOTON}?${params}`)).json();
    return datos.features
        .filter((f) => TIPOS_CIUDAD.includes(f.properties.osm_value))
        .filter((f) => !codigo || (f.properties.countrycode || '').toLowerCase() === codigo)
        .slice(0, 8)
        .map((f) => ({
            texto: f.properties.name,
            detalle: [f.properties.state, nombrePais(f.properties.countrycode) ||
f.properties.country].filter(Boolean).join(', '),
            codigo: (f.properties.countrycode || '').toLowerCase(),
            bandera: f.properties.countrycode ? bandera(f.properties.countrycode.toLowerCase()) : null,
            lat: f.geometry.coordinates[1],
            lon: f.geometry.coordinates[0],
        }));
}, (item) => {
    ciudad.value = item.texto;
    if (!pais.value) pais.value = nombrePais(item.codigo) || '';
    ponerMarcador(item.lat, item.lon, 6);
    estado.textContent = `📍 ${item.texto}, ${pais.value}`;
});
```

El autocompletar reutilizable (sirve para país y ciudad)

public/js/ubicacion.js línea 195

```
input.addEventListener('input', () => {
    clearTimeout(temporizador);
    const numero = ++consulta;
    temporizador = setTimeout(async () => {
        try {
            const resultado = await buscar(input.value);
            if (numero !== consulta) return; // llegó una respuesta vieja
            items = resultado;
            marcado = -1;
            dibujar();
```

```
        } catch (e) {
            cerrar();
        }
    }, espera);
});
```

Validación en el servidor y la bandera

app/service/ValidadorRegistro.php — línea 92

```php
/**
 * País (de la lista oficial) y ciudad obligatorios; las coordenadas del mapa
 * son opcionales pero deben tener sentido. Deja el país con su nombre oficial.
 */
public function validarUbicacion(array &$d): ?string
{
    if (($d['pais'] ?? '') === '' || ($d['ciudad'] ?? '') === '') {
        return 'Completá país y ciudad.';
    }
    $pais = $this->paises->nombreValido($d['pais']);
    if ($pais === null) {
        return 'Elegí un país de la lista.';
    }
    $d['pais'] = $pais;
    if (mb_strlen($d['ciudad']) > 80) {
        return 'El nombre de la ciudad es demasiado largo.';
    }

    $lat = $d['latitud'] ?? null;
    $lon = $d['longitud'] ?? null;
    if ($lat === null && $lon === null) {
        return null;
    }
    if (!is_numeric($lat) || !is_numeric($lon) || abs((float) $lat) > 90 || abs((float) $lon) > 180) {
        return 'La ubicación del mapa no es válida.';
    }
    return null;
}
```

app/service/PaisService.php — línea 16

```php
/** Devuelve el nombre oficial de la lista ("argentina" → "Argentina") o null si no existe. */
public function nombreValido(string $nombre): ?string
{
    $pais = $this->buscar($nombre);
    return $pais['nombre'] ?? null;
}
```

app/service/PaisService.php — línea 38

```php
/** "ar" → 🇦🇷 (cada letra se convierte en un "indicador regional" Unicode). */
public static function emoji(string $codigo): string
{
    $emoji = '';
    foreach (str_split(strtoupper($codigo)) as $letra) {
        $emoji .= mb_chr(0x1F1E6 + ord($letra) - ord('A'), 'UTF-8');
    }
    return $emoji;
}
```

El mapa del perfil

`PerfilService` redondea las coordenadas a 2 decimales (~1 km) porque el perfil es público: se muestra la zona, no la casa. Si el jugador no tocó el mapa al registrarse, `perfil.js` busca "ciudad, país" en Nominatim.

app/service/PerfilService.php — línea 65

```php
/**
 * Punto para el mapa del perfil. Las coordenadas se redondean a 2 decimales
 * (~1 km): el perfil es público y alcanza con mostrar la zona, no la casa.
 * Si no marcó el mapa al registrarse, se busca "ciudad, país" en el navegador.
 */
private static function puntoMapa(array $usuario): array
{
    $tieneCoordenadas = $usuario['latitud'] !== null && $usuario['longitud'] !== null;
    return [
        'lat' => $tieneCoordenadas ? round((float) $usuario['latitud'], 2) : '',
        'lon' => $tieneCoordenadas ? round((float) $usuario['longitud'], 2) : '',
        'buscar' => $usuario['ciudad'] . ', ' . $usuario['pais'],
    ];
}
```

public/js/perfil.js — línea 37

```javascript
// ── Mapa de la ubicación ──────────────────────
const caja = document.getElementById('mapa-perfil');
if (caja) {
    const sinMapa = (texto) => { caja.classList.add('mapa-sin-conexion'); caja.textContent = texto; };

    const mostrar = (lat, lon) => {
        const mapa = L.map(caja, { scrollWheelZoom: false, zoomControl: true }).setView([lat, lon], 10);
        L.tileLayer('https://tile.openstreetmap.org/{z}/{x}/{y}.png', {
            maxZoom: 18,
            attribution: '&copy; <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a>',
        }).addTo(mapa);
        const icono = L.divIcon({ className: 'marcador-zenit', html: '<span></span>', iconSize: [18, 18]
});
        L.marker([lat, lon], { icon: icono, keyboard: false }).addTo(mapa);
    };

    if (!window.L) {
        sinMapa('El mapa no cargó (¿sin internet?).');
    } else if (caja.dataset.lat && caja.dataset.lon) {
        mostrar(parseFloat(caja.dataset.lat), parseFloat(caja.dataset.lon));
    } else {
        // No marcó el mapa al registrarse: se busca la ciudad por nombre (Nominatim)
        fetch('https://nominatim.openstreetmap.org/search?format=json&limit=1&q=' +
encodeURIComponent(caja.dataset.buscar))
            .then((r) => r.json())
            .then((res) => (res[0] ? mostrar(parseFloat(res[0].lat), parseFloat(res[0].lon)) : sinMapa('No
encontramos la ciudad en el mapa.')))
            .catch(() => sinMapa('El mapa no cargó (¿sin internet?).'));
    }
}
```

¿Qué es vital? Leaflet y los mosaicos para ver el mapa; Nominatim y Photon para completar solos. Si alguno no carga (sin internet), el registro **igual funciona**: se puede escribir país y ciudad a mano y el servidor los valida con `paises.json`.

07 · JUEGO

EL BOTÓN JUGAR, PASO A PASO

Todo lo que pasa desde el clic hasta que aparece la ruleta.

1. **El botón es un link** a `/juego` (está en `homeView.mustache`). Si había una partida a medias, el lobby escribe **SEGUIR** en vez de JUGAR.
2. **¿Tiene sesión?** `JuegoController::show()` llama a `Session::requerirLogin()`. Si no inició sesión, guarda "juego" como destino y lo manda al login; después vuelve solo.
3. **Se dibuja la pantalla** `juegoView.mustache`: el HUD (vidas, letras, puntos), la ruleta vacía, la pantalla de pregunta, los modales de ronda, premio y reporte, y el fin. Se le pasan las categorías en JSON para la tira de muestra.
4. **Arranca juego.js**: muestra "cargando" y le pregunta al servidor `POST /juego/estado`.
5. **¿Había partida?** Si hay una en curso (por ejemplo, recargaste la página), sigue donde estaba. Si no hay o terminó, pide `POST /juego/iniciar`.
6. **El servidor crea la partida**: inserta una fila en `partida` y guarda en la sesión el estado inicial: 3 vidas, 0 letras, 0 puntos, Z al 1,5 %, fase "girar".
7. **juego.js dibuja según la fase**: como es "girar", muestra la ruleta y el botón GIRAR.

app/view/homeView.mustache — línea 21

```
<!-- Botón JUGAR con anillos -->
<div class="jugar-escena">
    <div class="anillo anillo-externo"></div>
    <div class="anillo anillo-interno"></div>
    <a href="{{base}}/juego" class="boton-jugar">
        <span>{{#partida_en_curso}}Seguir{{/partida_en_curso}}
{{^partida_en_curso}}Jugar{{/partida_en_curso}}</span>
    </a>
</div>
```

app/controller/JuegoController.php — línea 41

```
public function show(): void
{
    Session::requerirLogin('Iniciá sesión o creá una cuenta para jugar.');
    $this->render->renderiza('juego', [
        'nav_jugar' => true,
        'modo_juego' => true,
        // Opciones del formulario "Reportar un problema"
        'motivos_reporte' => array_map(
            fn($clave, $texto) => ['clave' => $clave, 'texto' => $texto],
            array_keys(ReporteService::MOTIVOS),
            ReporteService::MOTIVOS
        ),
        // Ítems de muestra para la ruleta antes del primer giro
        'categorias_json' => json_encode(
            array_map([RuletaService::class, 'paraTira'], $this->categorias->todas()),
            JSON_UNESCAPED_UNICODE
        ),
        'reglas' => [
            'vidas' => ReglasJuego::VIDAS_INICIALES,
            'preguntas' => ReglasJuego::PREGUNTAS_POR_RONDA,
```

```
            'minimo' => ReglasJuego::MINIMO_ACIERTOS,
            'rondas_letra' => ReglasJuego::RONDAS_PARA_LETRA,
            'segundos' => ReglasJuego::SEGUNDOS_POR_PREGUNTA,
        ],
    ]);
}
```

public/js/juego.js — línea 63

```
async function iniciar() {
    mostrarPantalla('cargando');
    try {
        let json = await api('estado');
        if (!json.estado || json.estado.fase === 'fin') {
            json = await api('iniciar');
        }
        aplicarEstado(json.estado);
    } catch (e) {
        $('juego').querySelector('.cargando').textContent = 'No se pudo cargar la partida: ' + e.message;
    }
}
```

public/js/juego.js — línea 83

```
function aplicarEstado(nuevo) {
    estado = nuevo;
    dibujarHud();

    switch (estado.fase) {
        case 'girar':    prepararRuleta(); break;
        case 'pregunta': cargarPregunta(); break;
        case 'premio':   abrirPremio(); break;
        case 'fin':      mostrarFin(); break;
    }
}
```

app/service/JuegoService.php — línea 56

```
public function iniciar(int $usuarioId): array
{
    $this->abandonar($usuarioId); // una partida a medias cuenta como derrota
    $estado = EstadoJuego::nuevo($this->partidas->crear($usuarioId), $usuarioId);
    $this->almacen->guardar($estado);
    return $this->publico($estado);
}
```

app/service/EstadoJuego.php — línea 31

```
/** Estado inicial de una partida nueva. */
public static function nuevo(int $partidaId, int $usuarioId): array
{
    return [
        'partida_id' => $partidaId,
        'usuario_id' => $usuarioId,
        'fase' => JuegoService::FASE_GIRAR,
        'vidas' => ReglasJuego::VIDAS_INICIALES,
        'letras' => '',                 // ej: "ZEN"
        'puntos' => 0,
        'respondidas' => 0,
        'correctas' => 0,
        'prob_z' => ReglasJuego::PROB_Z_BASE,
        'rondas_superadas' => 0,        // contador hacia la ronda de letra
        'origen_premio' => null,        // "letra" | "z"
        'resultado' => null,            // "victoria" | "derrota"
```

```
        'vistas' => [],                 // ids de preguntas ya usadas en esta partida
        'ronda' => null,                // ronda en juego (ver JuegoService::nuevaRonda)
        'ultima_ronda' => null,         // preguntas de la ronda terminada (las que se pueden reportar)
    ];
}
```

Cómo se comunican juego.js y el servidor

Todo el juego es una pequeña **API**: juego.js manda pedidos con `fetch()` y el servidor responde JSON. El controller exige POST y sesión, y transforma los errores en respuestas claras.

public/js/juego.js — línea 34

```
async function api(accion, datos = {}) {
    const respuesta = await fetch(`${BASE}/juego/${accion}`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(datos),
    });
    const json = await respuesta.json().catch(() => ({ ok: false, error: 'Respuesta inválida del servidor.'
}));

    if (respuesta.status === 401) {
        window.location.href = `${BASE}/login`;
        throw new Error(json.error);
    }
    if (!json.ok) {
        throw new Error(json.error || 'Error inesperado.');
    }
    return json;
}
```

app/controller/JuegoController.php — línea 119

```
/**
 * Envoltorio común de la API: exige POST y sesión, y transforma
 * las excepciones en respuestas JSON con el código HTTP adecuado.
 */
private function api(callable $accion): void
{
    if (!Request::esPost()) {
        Json::error('Método no permitido.', 405);
    }
    $usuarioId = Session::usuarioId();
    if ($usuarioId === null) {
        Json::error('Tu sesión expiró. Volvé a iniciar sesión.', 401);
    }

    try {
        Json::responder(['ok' => true] + $accion($usuarioId));
    } catch (DomainException $e) {
        Json::error($e->getMessage(), 409);
    } catch (Throwable $e) {
        Log::error('Juego: ' . $e->getMessage() . ' en ' . $e->getFile() . ':' . $e->getLine());
        Json::error('Ocurrió un error inesperado.', 500);
    }
}
```

> **Regla de oro de ZENIT:** el navegador solo **dibuja**. Las vidas, letras, puntos, la probabilidad de la Z y la respuesta correcta viven en el servidor. Aunque alguien toque el JavaScript, no puede hacer trampa.

08 · JUEGO

LAS FASES DE UNA PARTIDA

La partida es una "máquina de estados": siempre está en una fase y cada acción la mueve a otra.

FASE	QUÉ VE EL JUGADOR	QUÉ ACCIÓN ACEPTA EL SERVIDOR	PASA A
girar	La ruleta y el botón GIRAR	`girar`	pregunta o premio (si salió Z)
pregunta	Pregunta, 6 opciones y reloj	`pregunta` , `responder`	pregunta (sigue la ronda), girar, premio o fin
premio	Elegir letra o corazón	`premio`	girar o fin (si completó ZENIT)
fin	Resultado final	`iniciar` (partida nueva)	girar

Cada acción empieza con `exigirFase()` : si llega una acción que no corresponde (por ejemplo, responder cuando hay que girar), se rechaza con error 409. Así no se pueden saltear pasos.

app/service/JuegoService.php — línea 372

```php
private function exigirFase(int $usuarioId, string $fase): array
{
    $estado = $this->cargar($usuarioId);
    if ($estado === null || $estado['fase'] !== $fase) {
        throw new DomainException('Acción no válida en este momento de la partida.');
    }
    return $estado;
}
```

Dónde vive el estado: la sesión

`EstadoJuego` guarda la partida en `$_SESSION['zenit_partida']` , en el servidor. Si recargás la página, la partida sigue igual.

app/service/EstadoJuego.php — línea 16

```php
public function obtener(): ?array
{
    return $_SESSION[self::CLAVE] ?? null;
}
```

app/service/EstadoJuego.php — línea 21

```php
public function guardar(array $estado): void
{
    $_SESSION[self::CLAVE] = $estado;
}
```

Qué se le manda al navegador

`publico()` arma una copia "segura" del estado: nunca incluye la respuesta correcta ni los ids de las preguntas.

app/service/JuegoService.php — línea 401

```php
/** Lo que ve el navegador: nunca incluye respuestas ni ids de preguntas. */
private function publico(array $estado): array
{
    $letras = [];
    foreach (str_split(ReglasJuego::PALABRA) as $i => $letra) {
        $letras[] = ['letra' => $letra, 'obtenida' => $i < strlen($estado['letras'])];
    }

    $ronda = $estado['ronda'];
    return [
        'fase' => $estado['fase'],
        'vidas' => $estado['vidas'],
        'vidas_maximas' => ReglasJuego::VIDAS_MAXIMAS,
        'letras' => $letras,
        'cantidad_letras' => strlen($estado['letras']),
        'proxima_letra' => ReglasJuego::PALABRA[strlen($estado['letras'])] ?? null,
        'puntos' => $estado['puntos'],
        'respondidas' => $estado['respondidas'],
        'correctas' => $estado['correctas'],
        'prob_z' => round($estado['prob_z'] * 100, 1),
        'rondas_superadas' => $estado['rondas_superadas'],
        'rondas_para_letra' => ReglasJuego::RONDAS_PARA_LETRA,
        'minimo_aciertos' => ReglasJuego::MINIMO_ACIERTOS,
        'minimo_aciertos_letra' => ReglasJuego::MINIMO_ACIERTOS_LETRA,
        'origen_premio' => $estado['origen_premio'],
        'puede_elegir_vida' => $estado['vidas'] < ReglasJuego::VIDAS_MAXIMAS,
        'resultado' => $estado['resultado'],
        'ronda' => $ronda === null ? null : [
            'tipo' => $ronda['tipo'],
            'categoria' => $ronda['categoria'],
            'numero' => $ronda['indice'] + 1,
            'total' => count($ronda['preguntas']),
            'aciertos' => $ronda['aciertos'],
        ],
    ];
}
```

Todas las reglas, en un solo archivo

Cambiar el balance del juego es cambiar un número en `ReglasJuego.php`:

app/service/ReglasJuego.php — línea 9

```php
final class ReglasJuego
{
    public const PALABRA = 'ZENIT';             // letras a recolectar, en orden

    public const VIDAS_INICIALES = 3;
    public const VIDAS_MAXIMAS = 5;             // tope al elegir corazón como premio

    public const PREGUNTAS_POR_RONDA = 3;       // ronda de categoría y ronda de letra
    public const OPCIONES_POR_PREGUNTA = 6;     // 1 correcta + 5 incorrectas
    public const MINIMO_ACIERTOS = 2;           // en ronda de categoría: menos de 2 → -1 vida
    public const RONDAS_PARA_LETRA = 3;         // rondas superadas seguidas → ronda de letra
    public const MINIMO_ACIERTOS_LETRA = 2;     // en ronda de letra: 2 de 3 → letra o corazón

    public const SEGUNDOS_POR_PREGUNTA = 20;
    public const SEGUNDOS_DE_GRACIA = 2;        // margen por la demora de la red

    // Categoría Z: muy poco probable; cada acierto la hace un poquito más probable.
    public const PROB_Z_BASE = 0.015;           // 1,5 %
    public const PROB_Z_POR_ACIERTO = 0.001;    // +0,1 % por respuesta correcta
    public const PROB_Z_MAXIMA = 0.05;          // nunca más de 5 %

    // Puntos (suman al "elo" del perfil)
```

```
    public const PUNTOS_ACIERTO = 10;
    public const PUNTOS_RONDA_SUPERADA = 20;
    public const PUNTOS_LETRA = 100;
    public const PUNTOS_VICTORIA = 500;
}
```

09 · JUEGO

LA RULETA

El servidor decide qué sale; el navegador solo hace la animación.

Parte 1 · El servidor elige el resultado

1. Al tocar GIRAR, juego.js pide `POST /juego/girar`.
2. `JuegoService::girar()` revisa en este orden: **¿ya superó 3 rondas?** → sale sí o sí la **ronda de letra**. Si no, **¿sale la Z?** (sorteo con su probabilidad). Si no, **una categoría al azar** entre las 8.
3. Prepara la ronda (elige las 3 preguntas) y guarda el estado.
4. `RuletaService::armarTira()` arma una tira de **48 ítems**: el ganador va en la posición **40** y el resto son categorías al azar como relleno. Cada tanto mete una "Z" de relleno para dar suspenso, pero nunca pegada al ganador.
5. Responde: la tira, la posición del ganador, el resultado y el estado nuevo.

app/service/JuegoService.php — línea 64

```php
public function girar(int $usuarioId): array
{
    $estado = $this->exigirFase($usuarioId, self::FASE_GIRAR);
    $categorias = $this->categorias->todas();

    if ($estado['rondas_superadas'] >= ReglasJuego::RONDAS_PARA_LETRA) {
        $ganador = RuletaService::ITEM_LETRA;
        $estado['ronda'] = $this->rondaDeLetra($estado, $categorias);
        $estado['fase'] = self::FASE_PREGUNTA;
    } elseif ($this->ruleta->saleZ($estado['prob_z'])) {
        // ¡Categoría Z! Te ahorra las 3 rondas: premio directo.
        $ganador = RuletaService::ITEM_Z;
        $estado['prob_z'] = ReglasJuego::PROB_Z_BASE;
        $estado['rondas_superadas'] = 0;
        $estado['origen_premio'] = 'z';
        $estado['fase'] = self::FASE_PREMIO;
    } else {
        $categoria = $this->ruleta->categoriaAlAzar($categorias);
        $ganador = RuletaService::paraTira($categoria);
        $estado['ronda'] = $this->rondaDeCategoria($estado, $categoria);
        $estado['fase'] = self::FASE_PREGUNTA;
    }

    $this->almacen->guardar($estado);
    return [
        'tira' => $this->ruleta->armarTira($categorias, $ganador),
        'posicion' => RuletaService::POSICION_GANADOR,
        'resultado' => $ganador,
        'estado' => $this->publico($estado),
    ];
}
```

app/service/RuletaService.php — línea 27

```php
public function categoriaAlAzar(array $categorias): array
{
    return $categorias[array_rand($categorias)];
}
```

app/service/RuletaService.php línea 39

```php
/**
 * Tira para la animación. Incluye alguna "Z" de relleno para generar
 * suspenso, pero nunca pegada al ganador (así no confunde).
 */
public function armarTira(array $categorias, array $ganador): array
{
    $tira = [];
    for ($i = 0; $i < self::TOTAL_ITEMS; $i++) {
        $lejosDelGanador = abs($i - self::POSICION_GANADOR) > 2;
        $tira[] = ($lejosDelGanador && mt_rand(1, 14) === 1)
            ? self::ITEM_Z
            : self::paraTira($this->categoriaAlAzar($categorias));
    }
    $tira[self::POSICION_GANADOR] = $ganador;
    return $tira;
}
```

`array_rand()` elige un índice al azar: cada categoría tiene la misma chance (1 en 8). Las categorías nunca se eliminan del sorteo.

Parte 2 · El navegador anima de derecha a izquierda

1. juego.js dibuja los 48 ítems en una fila horizontal (`.ruleta-tira`, un **flex**).
2. La caja `.ruleta` tiene `overflow: hidden`: solo se ve una "ventana". En el centro hay una línea verde fija: el **marcador**.
3. Calcula cuánto mover la tira para que el ítem 40 quede bajo el marcador: `posición del ítem + medio ítem − media ventana`.
4. Le suma un **desvío al azar** de hasta ±30 % del ancho del ítem, para que no frene siempre en el centro exacto (más real).
5. Mueve la tira hacia la izquierda con `translateX(-destino)` y una transición de **5,2 segundos**. La curva `cubic-bezier(.08, .62, .12, 1)` hace que arranque rápido y frene despacio, como la caja de CS2.
6. Al terminar, ilumina el ganador, muestra el nombre (o el destello rojo si es la Z) y pasa a la fase siguiente.

public/js/juego.js línea 160

```js
async function girar() {
    const boton = $('btn-girar');
    boton.disabled = true;

    let json;
    try {
        json = await api('girar');
    } catch (e) {
        return sincronizar(e);
    }

    const tira = $('ruleta-tira');
    tira.style.transition = 'none';
    tira.style.transform = 'translateX(0)';
```

```
    tira.innerHTML = json.tira.map(htmlItem).join('');
    tira.dataset.usada = '1';

    // Cuánto hay que mover la tira para que el ganador quede bajo el marcador
    const item = tira.children[json.posicion];
    const ventana = $('ruleta').clientWidth;
    const azar = (Math.random() - 0.5) * item.offsetWidth * 0.6; // no siempre en el centro exacto
    const destino = item.offsetLeft + item.offsetWidth / 2 - ventana / 2 + azar;

    void tira.offsetWidth; // fuerza el reflow para que la transición arranque desde 0
    tira.style.transition = `transform ${DURACION_RULETA}ms cubic-bezier(.08, .62, .12, 1)`;
    tira.style.transform = `translateX(${-destino}px)`;

    await esperar(DURACION_RULETA + 150);
    item.classList.add('item-ganador');
    anunciarResultado(json.resultado);

    await esperar(1300);
    aplicarEstado(json.estado);
}
```

public/js/juego.js — línea 133

```
function htmlItem(item) {
    const clase = item.especial ? `item-${item.especial}` : '';
    return `<div class="item ${clase}" style="--c:${item.color}">
                <span class="item-emoji">${item.emoji}</span>
                <span class="item-nombre">${item.nombre}</span>
            </div>`;
}
```

public/css/zenit.css — línea 607

```
/* Ruleta (apertura de caja: los ítems pasan de derecha a izquierda) */
.ruleta {
    --ancho-item: 112px;
    --separacion: 10px;
    position: relative;
    width: 100%;
    height: 160px;
    overflow: hidden;
    background: var(--superficie);
    border: 1px solid var(--linea);
    border-radius: var(--radio);
    -webkit-mask-image: linear-gradient(90deg, transparent, #000 14%, #000 86%, transparent);
            mask-image: linear-gradient(90deg, transparent, #000 14%, #000 86%, transparent);
}
.ruleta-tira {
    display: flex;
    gap: var(--separacion);
    height: 100%;
    padding: 14px 0;
    will-change: transform;
}
.ruleta-marcador {
    position: absolute;
    top: 0;
    bottom: 0;
    left: 50%;
    z-index: 2;
    width: 3px;
    margin-left: -1.5px;
    background: var(--verde);
    box-shadow: 0 0 14px var(--verde);
}
```

```
.ruleta-marcador::before, .ruleta-marcador::after {
    content: '';
    position: absolute;
    left: 50%;
    margin-left: -9px;
    border: 9px solid transparent;
}
.ruleta-marcador::before { top: 0; border-top-color: var(--verde); }
.ruleta-marcador::after { bottom: 0; border-bottom-color: var(--verde); }
```

¿Por qué no frena "al azar" en el navegador? Porque si lo decidiera el JavaScript, cualquiera podría elegir su categoría. El azar real está en el servidor; la animación solo "llega" al resultado que ya se eligió.

10 · JUEGO

LA CATEGORÍA Z

Cómo sube su probabilidad y qué pasa cuando sale.

MOMENTO	PROBABILIDAD DE LA Z
Arranque de la partida	**1,5 %** (`PROB_Z_BASE`)
Cada respuesta correcta	**+0,1 %** (`PROB_Z_POR_ACIERTO`)
Tope	**5 %** como máximo (`PROB_Z_MAXIMA`)
Cuando sale la Z	Vuelve a **1,5 %** y el contador de rondas vuelve a 0

1. Cada vez que acertás, `registrarRespuesta()` suma 0,001 (0,1 %) a `prob_z` sin pasar de 0,05.
2. En cada giro, `saleZ()` saca un número al azar entre 0 y 1. Si es menor que la probabilidad, sale la Z. Ejemplo: con 2 % (0,02), sale si el número es menor a 0,02.
3. Si sale, **no hay preguntas**: va directo a la fase `premio` para elegir letra o corazón.
4. El HUD muestra la chance actual de la Z en vivo (ej: "2,3 %").

app/service/RuletaService.php — línea 21

```php
/** ¿Sale la categoría Z en este giro? */
public function saleZ(float $probabilidad): bool
{
    return mt_rand() / mt_getrandmax() < $probabilidad;
}
```

app/service/JuegoService.php — línea 321

```php
private function registrarRespuesta(array &$estado, int $categoriaId, bool $acerto): void
{
    $estado['respondidas']++;
    $this->estadisticas->registrarRespuesta($estado['usuario_id'], $categoriaId, $acerto);

    if ($acerto) {
        $estado['correctas']++;
        $estado['ronda']['aciertos']++;
        $estado['prob_z'] = min(ReglasJuego::PROB_Z_MAXIMA, $estado['prob_z'] +
ReglasJuego::PROB_Z_POR_ACIERTO);
        $this->sumarPuntos($estado, ReglasJuego::PUNTOS_ACIERTO);
    }
}
```

app/service/JuegoService.php — línea 73

```php
} elseif ($this->ruleta->saleZ($estado['prob_z'])) {
    // ¡Categoría Z! Te ahorra las 3 rondas: premio directo.
    $ganador = RuletaService::ITEM_Z;
    $estado['prob_z'] = ReglasJuego::PROB_Z_BASE;
    $estado['rondas_superadas'] = 0;
    $estado['origen_premio'] = 'z';
    $estado['fase'] = self::FASE_PREMIO;
```

Ejemplo: si acertaste 20 preguntas, la chance es 1,5 % + 20 × 0,1 % = **3,5 %** por giro.

11 · JUEGO

PREGUNTAS Y RELOJ

Cómo se eligen, cómo se mezclan las opciones y cómo se controla el tiempo.

Elegir las preguntas

Se eligen al azar entre las de la categoría que **todavía no salieron** en esta partida (lista `vistas`). Si se agotan, esa categoría se reinicia.

app/service/JuegoService.php — línea 297

```php
/** Elige preguntas no vistas en esta partida (si se agotan, se reinicia esa categoría). */
private function elegirPreguntas(array &$estado, int $categoriaId, int $cantidad): array
{
    $todas = $this->preguntas->idsPorCategoria($categoriaId);
    $disponibles = array_values(array_diff($todas, $estado['vistas']));

    if (count($disponibles) < $cantidad) {
        $estado['vistas'] = array_values(array_diff($estado['vistas'], $todas));
        $disponibles = $todas;
    }
    shuffle($disponibles);
    $elegidas = array_slice($disponibles, 0, $cantidad);
    if ($elegidas === []) {
        throw new DomainException('No hay preguntas cargadas para esta categoría.');
    }

    $estado['vistas'] = array_merge($estado['vistas'], $elegidas);
    return $elegidas;
}
```

Mostrar la pregunta (sin la respuesta)

1. juego.js pide `POST /juego/pregunta`.
2. La primera vez, el servidor **mezcla** las 6 opciones (`shuffle`) y guarda ese orden, para que no cambie si recargás.
3. Anota la hora exacta en que se mostró (`microtime`): el **reloj real está en el servidor**.
4. Manda el enunciado, las opciones con letras A–F (sin decir cuál es la correcta) y el tiempo restante.
5. juego.js dibuja el reloj circular de 20 segundos. Si llega a 0, responde "sin opción".

app/service/JuegoService.php — línea 96

```php
/** Pregunta actual SIN la respuesta correcta. Arranca el reloj la primera vez. */
public function pregunta(int $usuarioId): array
{
    $estado = $this->exigirFase($usuarioId, self::FASE_PREGUNTA);
    $ronda = &$estado['ronda'];
    $pregunta = $this->preguntaEnCurso($estado);

    if ($ronda['orden'] === null) {
        $ids = array_column($pregunta['opciones'], 'id');
        shuffle($ids); // las opciones salen mezcladas
        $ronda['orden'] = $ids;
    }
```

```
    if ($ronda['inicio'] === null) {
        $ronda['inicio'] = microtime(true);
    }
    $orden = $ronda['orden'];
    $transcurrido = microtime(true) - $ronda['inicio'];
    unset($ronda); // cortar la referencia antes de guardar
    $categoria = $this->categorias->buscar((int) $pregunta['categoria_id']);
    $this->almacen->guardar($estado);

    $textos = array_column($pregunta['opciones'], 'texto', 'id');
    $opciones = [];
    foreach ($orden as $i => $id) {
        $opciones[] = ['id' => (int) $id, 'letra' => self::LETRAS_OPCION[$i], 'texto' => $textos[$id]];
    }

    return [
        'enunciado' => $pregunta['enunciado'],
        'categoria' => RuletaService::paraTira($categoria),
        'opciones' => $opciones,
        'tiempo_total' => ReglasJuego::SEGUNDOS_POR_PREGUNTA,
        'tiempo_restante' => max(0, ReglasJuego::SEGUNDOS_POR_PREGUNTA - $transcurrido),
        'estado' => $this->publico($estado),
    ];
}
```

public/js/juego.js línea 263

```
function arrancarReloj(total, restante) {
    clearInterval(reloj);
    const arco = $('reloj-arco');
    const numero = $('reloj-numero');
    const circunferencia = 119.4; // 2·π·19
    const fin = Date.now() + restante * 1000;

    const pintar = () => {
        const quedan = Math.max(0, (fin - Date.now()) / 1000);
        numero.textContent = Math.ceil(quedan);
        arco.style.strokeDashoffset = circunferencia * (1 - quedan / total);
        $('reloj').classList.toggle('reloj-urgente', quedan <= 5);
        if (quedan <= 0) {
            clearInterval(reloj);
            responder(null); // se acabó el tiempo
        }
    };
    pintar();
    reloj = setInterval(pintar, 200);
}
```

Responder

1. Al tocar una opción, juego.js bloquea los botones (evita doble clic) y manda `POST /juego/responder` con el id elegido.
2. El servidor busca la correcta en la base y calcula si se pasó del tiempo (20 s + 2 s de margen por la red).
3. **Acertó** si respondió a tiempo y eligió la correcta. Se guarda en las estadísticas.
4. Avanza a la próxima pregunta. Si era la tercera, cierra la ronda (sección 12).
5. Recién **ahora** le dice al navegador cuál era la correcta, para pintarla de verde.

app/service/JuegoService.php línea 133

```php
/** @param int|null $opcionId null = se acabó el tiempo */
public function responder(int $usuarioId, ?int $opcionId): array
{
    $estado = $this->exigirFase($usuarioId, self::FASE_PREGUNTA);
    $ronda = &$estado['ronda'];
    if ($ronda['inicio'] === null) {
        throw new DomainException('Primero hay que pedir la pregunta.');
    }

    $pregunta = $this->preguntaEnCurso($estado);
    $idCorrecta = $this->idOpcionCorrecta($pregunta);
    $fueraDeTiempo = microtime(true) - $ronda['inicio']
        > ReglasJuego::SEGUNDOS_POR_PREGUNTA + ReglasJuego::SEGUNDOS_DE_GRACIA;
    $acerto = !$fueraDeTiempo && $opcionId === $idCorrecta;

    $this->registrarRespuesta($estado, (int) $pregunta['categoria_id'], $acerto);

    // Avanzar a la siguiente pregunta de la ronda
    $ronda['indice']++;
    $ronda['inicio'] = null;
    $ronda['orden'] = null;
    unset($ronda); // cortar la referencia antes de reemplazar la ronda

    $resumen = null;
    if ($estado['ronda']['indice'] >= count($estado['ronda']['preguntas'])) {
        $resumen = $this->cerrarRonda($estado);
    }

    $this->almacen->guardar($estado);
    return [
        'correcta' => $acerto,
        'fuera_de_tiempo' => $fueraDeTiempo,
        'opcion_correcta' => $idCorrecta,
        'resumen_ronda' => $resumen,
        'estado' => $this->publico($estado),
    ];
}
```

public/js/juego.js línea 284

```javascript
async function responder(opcionId) {
    if (respondiendo) return;
    respondiendo = true;
    clearInterval(reloj);

    const botones = Array.from($('opciones').children);
    botones.forEach((b) => (b.disabled = true));

    let json;
    try {
        json = await api('responder', { opcion_id: opcionId });
    } catch (e) {
        return sincronizar(e);
    }

    // Pintar correcta / incorrecta
    botones.forEach((b) => {
        const id = Number(b.dataset.id);
        if (id === json.opcion_correcta) b.classList.add('opcion-correcta');
        else if (id === opcionId) b.classList.add('opcion-incorrecta');
        else b.classList.add('opcion-apagada');
    });
    if (json.fuera_de_tiempo || opcionId === null) {
        $('pregunta-ronda').textContent = '⏱ ¡Se acabó el tiempo!';
```

```
    }

    const vidaPerdida = json.resumen_ronda && json.resumen_ronda.vida_perdida;
    estado = json.estado;
    dibujarHud(vidaPerdida);

    await esperar(ESPERA_FEEDBACK);

    if (json.resumen_ronda) {
        ultimaRonda = { preguntas: json.resumen_ronda.preguntas, reportada: false };
        mostrarResumenRonda(json.resumen_ronda);
    } else {
        cargarPregunta();
    }
}
```

12 · JUEGO

VIDAS Y FIN DE RONDA

Qué decide el servidor cuando terminás las 3 preguntas.

TIPO DE RONDA	SI ACERTÁS 2 O 3	SI ACERTÁS 0 O 1
Categoría	Ronda superada: +20 puntos y suma 1 al contador de rondas (de 3).	**Perdés 1 vida.** El contador no se reinicia.
Letra	Vas al premio: letra o corazón.	Se escapa la letra (no perdés vida).

1. `cerrarRonda()` cuenta los aciertos de la ronda.
2. Si era de categoría y no llegó a 2: `$estado['vidas']--`. Si llegó: `rondas_superadas++`.
3. Si `rondas_superadas` llega a 3, avisa "Desbloqueaste una ronda de letra": el **próximo giro es sí o sí la ronda de letra** (sección 09).
4. Guarda las 3 preguntas en `ultima_ronda`: son las únicas que se pueden reportar ahora.
5. Si las vidas llegan a 0, termina la partida como **derrota**. Si no, vuelve a la fase girar (o premio).
6. juego.js muestra el modal con el resultado y anima el corazón que se pierde.

app/service/JuegoService.php — línea 238

```php
/** Evalúa la ronda terminada y devuelve un resumen para mostrar. */
private function cerrarRonda(array &$estado): array
{
    $ronda = $estado['ronda'];
    $total = count($ronda['preguntas']);
    $resumen = ['tipo' => $ronda['tipo'], 'aciertos' => $ronda['aciertos'], 'total' => $total,
                'vida_perdida' => false, 'letra_desbloqueada' => false, 'premio' => false];

    if ($ronda['tipo'] === 'categoria') {
        $resumen['superada'] = $ronda['aciertos'] >= ReglasJuego::MINIMO_ACIERTOS;
        if ($resumen['superada']) {
            $estado['rondas_superadas']++;
            $this->sumarPuntos($estado, ReglasJuego::PUNTOS_RONDA_SUPERADA);
            $resumen['letra_desbloqueada'] = $estado['rondas_superadas'] >= ReglasJuego::RONDAS_PARA_LETRA;
        } else {
            $estado['vidas']--;
            $resumen['vida_perdida'] = true;
        }
    } else {
        // Ronda de letra: alcanza con 2 de 3. Pase lo que pase, el contador vuelve a 0.
        $resumen['superada'] = $ronda['aciertos'] >= ReglasJuego::MINIMO_ACIERTOS_LETRA;
        $estado['rondas_superadas'] = 0;
        if ($resumen['superada']) {
            $estado['origen_premio'] = 'letra';
            $estado['fase'] = self::FASE_PREMIO;
            $resumen['premio'] = true;
        }
    }

    // Se guardan las preguntas de esta ronda: son las únicas que se pueden reportar ahora.
    $estado['ultima_ronda'] = ['preguntas' => $ronda['preguntas'], 'reportada' => false];
    $resumen['preguntas'] = $this->preguntasParaReporte($ronda['preguntas']);
```

```
    $estado['ronda'] = null;
    if ($estado['vidas'] <= 0) {
        $this->finalizar($estado, 'derrota');
    } elseif ($estado['fase'] !== self::FASE_PREMIO) {
        $estado['fase'] = self::FASE_GIRAR;
    }
    return $resumen;
}
```

public/js/juego.js — línea 99

```
function dibujarHud(vidaPerdida = false) {
    // Corazones: siempre se ven al menos 3 (los vacíos son vidas perdidas)
    const totalCorazones = Math.max(3, estado.vidas);
    $('hud-vidas').innerHTML = Array.from({ length: totalCorazones }, (_, i) => {
        const lleno = i < estado.vidas;
        const perdido = vidaPerdida && i === estado.vidas;
        return `<span class="corazon ${lleno ? '' : 'corazon-vacio'} ${perdido ? 'corazon-perdido' : ''}">♥
</span>`;
    }).join('');

    $('hud-letras').innerHTML = estado.letras
        .map((l) => `<span class="letra ${l.obtenida ? 'letra-obtenida' : ''}">${l.letra}</span>`)
        .join('');

    $('hud-puntos').textContent = estado.puntos.toLocaleString('es-AR');
    $('hud-z').textContent = `${estado.prob_z.toLocaleString('es-AR')}%`;

    $('hud-pasos').innerHTML = Array.from({ length: estado.rondas_para_letra }, (_, i) =>
        `<span class="paso ${i < estado.rondas_superadas ? 'paso-ok' : ''}"></span>`
    ).join('');
    $('hud-progreso-texto').textContent = estado.rondas_superadas >= estado.rondas_para_letra
        ? '¡Ronda de letra lista!'
        : 'Rondas para la próxima letra';
}
```

13 · JUEGO

RONDA DE LETRA Y PREMIO

Cómo se completan las letras Z·E·N·I·T.

1. Con 3 rondas superadas, `girar()` no sortea: arma la **ronda de letra** con `rondaDeLetra()`.
2. Elige **3 categorías distintas** (mezcla las 8 y toma las 3 primeras) y una pregunta de cada una.
3. Si acertás 2 de 3 → fase **premio**. Pase lo que pase, el contador de rondas vuelve a 0.
4. En el premio elegís: **letra** (se agrega la siguiente de ZENIT y suma 100 puntos) o **corazón** (+1 vida, máximo 5).
5. Si con esa letra se completa **ZENIT**, la partida termina en **victoria**.

app/service/JuegoService.php — línea 215

```php
/** 3 preguntas de 3 categorías distintas (1 de cada una). */
private function rondaDeLetra(array &$estado, array $categorias): array
{
    $ids = [];
    foreach ($this->ruleta->categoriasDistintas($categorias, ReglasJuego::PREGUNTAS_POR_RONDA) as
$categoria) {
        $ids[] = $this->elegirPreguntas($estado, (int) $categoria['id'], 1)[0];
    }
    return self::nuevaRonda('letra', null, $ids);
}
```

app/service/RuletaService.php — línea 32

```php
/** N categorías distintas (para la ronda de letra: 1 pregunta de cada una). */
public function categoriasDistintas(array $categorias, int $cantidad): array
{
    shuffle($categorias);
    return array_slice($categorias, 0, $cantidad);
}
```

app/service/JuegoService.php — línea 171

```php
/** @param string $eleccion "letra" | "vida" */
public function elegirPremio(int $usuarioId, string $eleccion): array
{
    $estado = $this->exigirFase($usuarioId, self::FASE_PREMIO);

    if ($eleccion === 'letra') {
        $estado['letras'] .= ReglasJuego::PALABRA[strlen($estado['letras'])];
        $this->sumarPuntos($estado, ReglasJuego::PUNTOS_LETRA);
    } elseif ($eleccion === 'vida' && $estado['vidas'] < ReglasJuego::VIDAS_MAXIMAS) {
        $estado['vidas']++;
    } else {
        throw new DomainException('Elección de premio inválida.');
    }

    $estado['origen_premio'] = null;
    $estado['fase'] = self::FASE_GIRAR;
    if ($estado['letras'] === ReglasJuego::PALABRA) {
        $this->finalizar($estado, 'victoria');
    }

    $this->almacen->guardar($estado);
```

```
    return ['estado' => $this->publico($estado)];
}
```

¿Cómo sabe qué letra sigue? Usa la palabra como una lista: si ya tenés "ZE" (2 letras), la próxima es `"ZENIT"[2]` = **N**.

14 · PERFIL

PUNTOS (ELO) Y ESTADÍSTICAS

Qué suma puntos, dónde se guarda cada número y cuándo.

ACCIÓN	PUNTOS	DÓNDE ESTÁ EN EL CÓDIGO
Respuesta correcta	+10	`registrarRespuesta()`
Ronda superada	+20	`cerrarRonda()`
Letra elegida	+100	`elegirPremio()`
Victoria (ZENIT)	+500	`finalizar()`

El **elo del perfil** es la suma de todos los puntos de todas tus partidas: la columna `puntos` de la tabla `usuario`. Se guarda **al instante** en cada acierto (si se corta la luz, no se pierden).

app/service/JuegoService.php — línea 334

```php
/** Los puntos se guardan al instante: si se corta la luz, no se pierden. */
private function sumarPuntos(array &$estado, int $puntos): void
{
    $estado['puntos'] += $puntos;
    $this->estadisticas->sumarPuntos($estado['usuario_id'], $puntos);
}
```

Cada respuesta actualiza dos lugares

1. **Totales del jugador** (tabla `usuario`): `respondidas + 1` y, si acertó, `correctas + 1`.
2. **Por categoría** (tabla `usuario_categoria`): si es la primera vez en esa categoría crea la fila; si no, la suma. De acá sale el % de acierto por categoría del perfil.
3. Al terminar la partida: `partidas_jugadas + 1` y, si ganó, `partidas_ganadas + 1`; además se cierra la fila de la tabla `partida` (historial).

app/model/EstadisticaModel.php — línea 17

```php
/** Suma una respuesta al total del usuario y a la categoría. */
public function registrarRespuesta(int $usuarioId, int $categoriaId, bool $correcta): void
{
    $this->actualizarTotales($usuarioId, $correcta);
    $this->actualizarCategoria($usuarioId, $categoriaId, $correcta);
}
```

app/model/EstadisticaModel.php — línea 24

```php
public function sumarPuntos(int $usuarioId, int $puntos): void
{
    $this->db->execute('UPDATE usuario SET puntos = puntos + ? WHERE id = ?', [$puntos, $usuarioId]);
}
```

app/model/EstadisticaModel.php — línea 62

```php
private function actualizarCategoria(int $usuarioId, int $categoriaId, bool $correcta): void
{
    $existe = $this->db->queryOne(
```

```
        'SELECT usuario_id FROM usuario_categoria WHERE usuario_id = ? AND categoria_id = ?',
        [$usuarioId, $categoriaId]
    );

    if ($existe === null) {
        $this->db->execute(
            'INSERT INTO usuario_categoria (usuario_id, categoria_id, respondidas, correctas) VALUES (?, ?,
1, ?)',
            [$usuarioId, $categoriaId, $correcta ? 1 : 0]
        );
        return;
    }

    $this->db->execute(
        'UPDATE usuario_categoria SET respondidas = respondidas + 1, correctas = correctas + ?
         WHERE usuario_id = ? AND categoria_id = ?',
        [$correcta ? 1 : 0, $usuarioId, $categoriaId]
    );
}
```

app/model/EstadisticaModel.php línea 29

```
public function registrarPartidaTerminada(int $usuarioId, bool $ganada): void
{
    $this->db->execute(
        'UPDATE usuario SET partidas_jugadas = partidas_jugadas + 1, partidas_ganadas = partidas_ganadas +
?
         WHERE id = ?',
        [$ganada ? 1 : 0, $usuarioId]
    );
}
```

Cómo se leen para el perfil

Un `LEFT JOIN` trae las 8 categorías aunque el jugador nunca haya jugado alguna (en ese caso, 0).

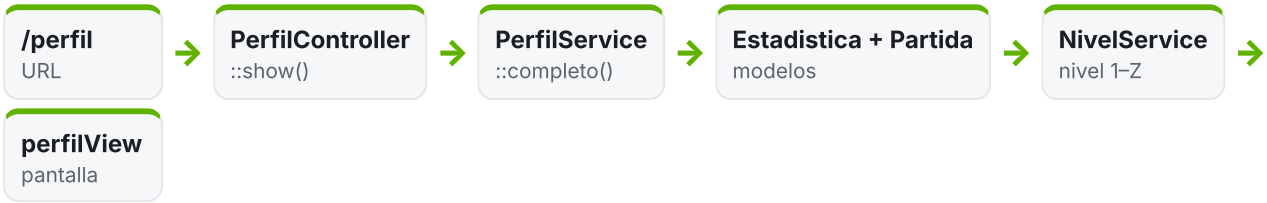
app/model/EstadisticaModel.php línea 38

```
/** Todas las categorías con lo respondido por el usuario (0 si nunca jugó esa). */
public function porCategoria(int $usuarioId): array
{
    return $this->db->queryAll(
        'SELECT c.id, c.slug, c.nombre, c.emoji, c.color,
                COALESCE(uc.respondidas, 0) AS respondidas,
                COALESCE(uc.correctas, 0)   AS correctas
         FROM categoria c
         LEFT JOIN usuario_categoria uc ON uc.categoria_id = c.id AND uc.usuario_id = ?
         ORDER BY c.id',
        [$usuarioId]
    );
}
```

15 · PERFIL

EL PERFIL Y EL NIVEL

De la base de datos a la pantalla estilo Steam.



1. `PerfilController::show()` exige sesión y busca al usuario.
2. `PerfilService::completo()` junta todo "listo para mostrar": nombre, foto, bandera, elo, % de acierto, partidas, % por categoría, últimas 5 partidas, el punto del mapa y la URL pública para el QR.
3. `NivelService::calcular()` decide el nivel con **dos condiciones**: cantidad mínima de respuestas correctas **y** % de acierto mínimo. También calcula qué falta para el siguiente.
4. La vista `perfilView.mustache` lo dibuja. El perfil público de otro jugador (`/perfil/ver?u=nombre`) usa lo mismo, pero sin el menú ⋮.

app/controller/PerfilController.php — línea 52

```php
public function show(): void
{
    $usuario = $this->usuarioLogueado();
    $this->render->renderiza('perfil', [
        'nav_perfil' => true,
        'es_propio' => true,
        'perfil' => $this->perfiles->completo($usuario),
    ]);
}
```

app/service/PerfilService.php — línea 25

```php
/** Datos básicos (barra superior, chips de usuario). */
public function resumen(array $usuario): array
{
    $respondidas = (int) $usuario['respondidas'];
    $correctas = (int) $usuario['correctas'];

    return [
        'id' => (int) $usuario['id'],
        'nombre_usuario' => $usuario['nombre_usuario'],
        'nombre_completo' => $usuario['nombre_completo'] ?? '',
        'iniciales' => strtoupper(substr($usuario['nombre_usuario'], 0, 2)),
        'foto' => $usuario['foto_perfil'] ? Url::to($usuario['foto_perfil']) : null,
        'pais' => $usuario['pais'],
        'bandera' => $this->paises->bandera($usuario['pais']), // imagen + emoji de la bandera
        'ciudad' => $usuario['ciudad'],
        'elo' => number_format((int) $usuario['puntos'], 0, ',', '.'),
        'nivel' => $this->niveles->calcular($respondidas, $correctas),
    ];
}
```

app/service/PerfilService.php — línea 45

```php
/** Todo lo del perfil completo. */
public function completo(array $usuario): array
{
    $respondidas = (int) $usuario['respondidas'];
    $correctas = (int) $usuario['correctas'];

    return $this->resumen($usuario) + [
        'miembro_desde' => date('m/Y', strtotime($usuario['creado'] ?? 'now')),
        'respondidas' => $respondidas,
        'correctas' => $correctas,
        'acierto' => NivelService::porcentaje($correctas, $respondidas),
        'partidas_jugadas' => (int) $usuario['partidas_jugadas'],
        'partidas_ganadas' => (int) $usuario['partidas_ganadas'],
        'categorias' => $this->categorias((int) $usuario['id']),
        'historial' => $this->historial((int) $usuario['id']),
        'mapa' => self::puntoMapa($usuario),
        'url_publica' => Url::absoluta('perfil/ver?u=' . rawurlencode($usuario['nombre_usuario'])),
    ];
}
```

app/service/NivelService.php — línea 17

```php
/** [código, correctas mínimas, % de acierto mínimo, color de marca] */
private const NIVELES = [
    ['1', 0,    0,  'blanco'],
    ['2', 20,   35, 'verde'],
    ['3', 50,   40, 'verde'],
    ['4', 100,  45, 'amarillo'],
    ['5', 200,  50, 'amarillo'],
    ['6', 350,  55, 'amarillo'],
    ['7', 500,  60, 'naranja'],
    ['8', 750,  65, 'naranja'],
    ['9', 1000, 70, 'rojo'],
    ['Z', 1500, 75, 'z'],
```

app/service/NivelService.php — línea 31

```php
public function calcular(int $respondidas, int $correctas): array
{
    if ($respondidas === 0) {
        return [
            'codigo' => '–', 'color' => 'gris', 'es_z' => false, 'sin_nivel' => true,
            'progreso' => 0, 'siguiente' => 'Jugá tu primera partida para obtener el nivel 1',
        ];
    }

    $acierto = self::porcentaje($correctas, $respondidas);
    $indice = 0;
    foreach (self::NIVELES as $i => [$codigo, $minCorrectas, $minAcierto]) {
        if ($correctas >= $minCorrectas && $acierto >= $minAcierto) {
            $indice = $i;
        }
    }

    [$codigo, , , $color] = self::NIVELES[$indice];
    return [
        'codigo' => $codigo,
        'color' => $color,
        'es_z' => $codigo === 'Z',
        'sin_nivel' => false,
    ] + $this->progresoAlSiguiente($indice, $correctas, $acierto);
}
```

Ejemplo: con 230 correctas y 52 % de acierto sos **nivel 5** (necesita 200 correctas y 50 %). Para el 6 te faltan 120 correctas y 55 % de acierto.

16 · PERFIL

EL CÓDIGO QR

Un QR que abre tu perfil público en cualquier celular.

1. El servidor arma la URL completa del perfil público con `Url::absoluta()`, por ejemplo `https://zenit.free.nf/perfil/ver?u=demo`. Se calcula sola con el dominio actual: funciona igual en XAMPP y en InfinityFree.
2. La vista la pone en el atributo `data-url` del `<div id="qr">` y carga la librería **qrcodejs** desde un CDN.
3. `perfil.js` crea el QR con `new QRCode(...)`: 260×260, colores de la marca y nivel de corrección M (se puede leer aunque esté un poco tapado).
4. Si la librería no cargó, usa de respaldo una imagen generada por `api.qrserver.com`. Si tampoco hay internet, avisa "QR no disponible".
5. El botón **Copiar enlace** copia la misma URL al portapapeles.

helper/Url.php — línea 23

```php
/** URL completa con dominio (la usa el QR del perfil). */
public static function absoluta(string $ruta = ''): string
{
    $https = !empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off';
    $host = $_SERVER['HTTP_HOST'] ?? 'localhost';
    return ($https ? 'https' : 'http') . '://' . $host . self::to($ruta);
}
```

app/service/PerfilService.php — línea 61

```php
'url_publica' => Url::absoluta('perfil/ver?u=' . rawurlencode($usuario['nombre_usuario'])),
```

public/js/perfil.js — línea 65

```javascript
// ── QR ──────────────────────────────────────────
const qr = document.getElementById('qr');
if (qr) {
    if (typeof QRCode !== 'undefined') {
        new QRCode(qr, {
            text: qr.dataset.url,
            width: 260,
            height: 260,
            colorDark: '#121417',
            colorLight: '#F2F3F5',
            correctLevel: QRCode.CorrectLevel.M,
        });
        qr.removeAttribute('title'); // qrcodejs pone la URL como tooltip
    } else {
        // Respaldo si el CDN de la librería no cargó: imagen generada por api.qrserver.com
        const img = document.createElement('img');
        img.alt = 'Código QR del perfil';
        img.src = 'https://api.qrserver.com/v1/create-qr-code/?size=260x260&margin=0&data='
            + encodeURIComponent(qr.dataset.url);
        img.onerror = () => { qr.classList.add('qr-sin-conexion'); qr.textContent = 'QR no disponible sin
conexión'; };
        qr.appendChild(img);
    }
```

```
}

// ── Copiar enlace ──────────────────────────────
const copiar = document.getElementById('btn-copiar');
if (copiar) {
    copiar.addEventListener('click', async function () {
        try {
            await navigator.clipboard.writeText(copiar.dataset.url);
            copiar.textContent = '¡Copiado!';
        } catch (e) {
            window.prompt('Copiá el enlace:', copiar.dataset.url);
        }
        setTimeout(() => (copiar.textContent = 'Copiar enlace'), 2000);
    });
```

17 · JUEGO

REPORTAR PREGUNTAS

Del link "Reportar un problema" a la tabla reporte.

1. Al terminar una ronda, el servidor manda el resumen con las **3 preguntas** (número, enunciado, categoría) y guarda sus ids en `ultima_ronda`.
2. Debajo de **Continuar** aparece "⚠ Reportar un problema". Al tocarlo se abre el modal con las 3 preguntas numeradas para tildar.
3. El jugador tilda una o más, elige un **motivo** (ofensiva, respuesta incorrecta, confusa u otro) y puede escribir un comentario.
4. juego.js revisa que haya al menos una pregunta y un motivo, y manda `POST /juego/reportar` con los números (1, 2, 3), no los ids.
5. `ReporteService` valida: que haya una ronda recién terminada, que no se haya reportado ya, que el motivo exista y que los números sean válidos. Recorta el comentario a 500 caracteres.
6. Por cada pregunta tildada, `ReporteModel` inserta una fila en `reporte` con estado **pendiente**.
7. Marca la ronda como reportada (un reporte por ronda) y el link cambia a "✓ Reporte enviado".

¿A dónde va? Queda en la tabla `reporte` de MySQL. Hoy se ve desde phpMyAdmin. El **Panel de administrador** (próxima entrega) los va a listar para revisarlos, editarlos o desactivar la pregunta.

public/js/juego.js — línea 432

```
async function enviarReporte(e) {
    e.preventDefault();
    const form = $('form-reporte');
    const preguntas = [...form.querySelectorAll('.reporte-pregunta input:checked')].map((c) =>
Number(c.value));
    const motivo = (form.querySelector('input[name=motivo]:checked') || {}).value;
    const error = $('reporte-error');

    if (!preguntas.length) { error.textContent = 'Tildá al menos una pregunta.'; error.hidden = false;
return; }
    if (!motivo) { error.textContent = 'Elegí un motivo.'; error.hidden = false; return; }

    $('reporte-enviar').disabled = true;
    try {
        await api('reportar', { preguntas, motivo, detalle: $('reporte-detalle').value });
        ultimaRonda.reportada = true;
        cerrarReporte();
    } catch (err) {
        error.textContent = err.message;
        error.hidden = false;
    } finally {
        $('reporte-enviar').disabled = false;
    }
}
```

app/service/ReporteService.php — línea 31

```
/**
 * @param int[]  $numeros posiciones de las preguntas en la ronda (1, 2, 3)
 * @return int cantidad de preguntas reportadas
```

```php
 */
public function reportar(int $usuarioId, array $numeros, string $motivo, string $detalle): int
{
    $estado = $this->almacen->obtener();
    $ronda = $estado['ultima_ronda'] ?? null;

    if (!$estado || $estado['usuario_id'] !== $usuarioId || $ronda === null) {
        throw new DomainException('Solo podés reportar al terminar una ronda.');
    }
    if ($ronda['reportada']) {
        throw new DomainException('Ya enviaste un reporte de esta ronda.');
    }
    if (!isset(self::MOTIVOS[$motivo])) {
        throw new DomainException('Elegí un motivo.');
    }

    $numeros = array_unique(array_map('intval', $numeros));
    $validos = array_filter($numeros, fn($n) => $n >= 1 && $n <= count($ronda['preguntas']));
    if ($validos === [] || count($validos) !== count($numeros)) {
        throw new DomainException('Elegí al menos una pregunta.');
    }

    $detalle = trim($detalle);
    $detalle = $detalle === '' ? null : mb_substr($detalle, 0, self::MAX_DETALLE);
    foreach ($validos as $numero) {
        $this->reportes->crear($ronda['preguntas'][$numero - 1], $usuarioId, $motivo, $detalle);
    }

    $estado['ultima_ronda']['reportada'] = true; // un reporte por ronda
    $this->almacen->guardar($estado);
    return count($validos);
}
```

app/model/ReporteModel.php — línea 16

```php
public function crear(int $preguntaId, int $usuarioId, string $motivo, ?string $detalle): void
{
    $this->db->execute(
        'INSERT INTO reporte (pregunta_id, usuario_id, motivo, detalle) VALUES (?, ?, ?, ?)',
        [$preguntaId, $usuarioId, $motivo, $detalle]
    );
}
```

app/sql/zenit.sql — línea 100

```sql
CREATE TABLE reporte (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    pregunta_id INT NOT NULL,
    usuario_id  INT NOT NULL,
    motivo      ENUM('ofensiva','respuesta_incorrecta','confusa','otro') NOT NULL,
    detalle     VARCHAR(500) NULL,
    estado      ENUM('pendiente','revisado','descartado') NOT NULL DEFAULT 'pendiente',
    creado      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (pregunta_id) REFERENCES pregunta(id) ON DELETE CASCADE,
    FOREIGN KEY (usuario_id)  REFERENCES usuario(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

Se mandan **números** (1, 2, 3) y no ids de pregunta: el servidor los traduce con la lista que él mismo guardó. Así nadie puede reportar preguntas que no le tocaron.

18 · JUEGO

FIN DE LA PARTIDA

Qué significa terminar y qué queda guardado.

CÓMO TERMINA	RESULTADO	PUNTOS EXTRA
Completaste Z·E·N·I·T	**victoria**	+500
Te quedaste sin vidas	**derrota**	—
Empezaste otra partida dejando una a medias	**derrota** (abandono)	—

1. `finalizar()` pone la fase en **fin** y guarda el resultado.
2. Si ganó, suma los 500 puntos.
3. `PartidaModel::cerrar()` completa la fila de la partida: resultado, puntos, letras, respondidas, correctas y fecha de fin. Así aparece en el **historial del perfil**.
4. `registrarPartidaTerminada()` suma 1 a partidas jugadas (y a ganadas si corresponde).
5. juego.js muestra la pantalla final: "¡ZENIT!" o "Sin vidas", las letras, los puntos y las correctas, con el botón para jugar otra.
6. Al tocar **Jugar otra**, `iniciar()` crea una partida nueva desde cero.

app/service/JuegoService.php — línea 341

```php
private function finalizar(array &$estado, string $resultado): void
{
    if ($resultado === 'victoria') {
        $this->sumarPuntos($estado, ReglasJuego::PUNTOS_VICTORIA);
    }
    $estado['fase'] = self::FASE_FIN;
    $estado['resultado'] = $resultado;
    $estado['ronda'] = null;

    $this->partidas->cerrar(
        $estado['partida_id'],
        $resultado,
        $estado['puntos'],
        $estado['letras'],
        $estado['respondidas'],
        $estado['correctas']
    );
    $this->estadisticas->registrarPartidaTerminada($estado['usuario_id'], $resultado === 'victoria');
}
```

app/model/PartidaModel.php — línea 22

```php
public function cerrar(int $id, string $resultado, int $puntos, string $letras, int $respondidas, int
$correctas): void
{
    $this->db->execute(
        'UPDATE partida
         SET resultado = ?, puntos = ?, letras = ?, respondidas = ?, correctas = ?, terminada = ?
         WHERE id = ?',
        [$resultado, $puntos, $letras, $respondidas, $correctas, date('Y-m-d H:i:s'), $id]
```

```
    );
}
```

app/service/JuegoService.php — línea 195

```php
/** Termina la partida en curso (cuenta como derrota) y limpia la sesión. */
public function abandonar(int $usuarioId): void
{
    $estado = $this->cargar($usuarioId);
    if ($estado && $estado['fase'] !== self::FASE_FIN) {
        $this->finalizar($estado, 'derrota');
    }
    $this->almacen->borrar();
}
```

public/js/juego.js — línea 459

```javascript
function mostrarFin() {
    const gano = estado.resultado === 'victoria';
    const fin = $('fin');
    fin.classList.toggle('fin-victoria', gano);
    fin.classList.toggle('fin-derrota', !gano);

    $('fin-sobretitulo').textContent = gano ? 'Llegaste a tu zenit' : 'Fin de la partida';
    $('fin-titulo').textContent = gano ? '¡ZENIT!' : 'Sin vidas';
    $('fin-letras').innerHTML = $('hud-letras').innerHTML;
    $('fin-puntos').textContent = estado.puntos.toLocaleString('es-AR');
    $('fin-correctas').textContent = `${estado.correctas}/${estado.respondidas}`;
    $('fin-cant-letras').textContent = `${estado.cantidad_letras}/5`;
    mostrarPantalla('fin');
}
```

19 · REFERENCIA

QUÉ ARCHIVO HACE QUÉ

Para encontrar rápido dónde está cada cosa.

FUNCIONALIDAD	ARCHIVOS PRINCIPALES
Entrada y rutas	`.htaccess`, `index.php`, `helper/Router.php`, `app/Configuration.php`
Base de datos	`app/sql/zenit.sql`, `helper/MyDatabase.php`, `config/config.ini`
Registro	`RegistroController`, `ValidadorRegistro`, `UsuarioModel`, `registroView`
Código por mail	`VerificacionService`, `CorreoService`, `helper/SmtpMailer.php`, `verificarView`
Login y sesión	`LoginController`, `helper/Session.php`, `loginView`
Mapa y país/ciudad	`public/js/ubicacion.js`, `partials/ubicacion`, `PaisService`, `public/data/paises.json`
Lobby (botón JUGAR)	`HomeController`, `homeView`
Juego (reglas)	`JuegoService`, `ReglasJuego`, `EstadoJuego`, `RuletaService`
Juego (pantalla)	`JuegoController`, `juegoView`, `public/js/juego.js`
Preguntas	`PreguntaModel`, `CategoriaModel`
Puntos y estadísticas	`EstadisticaModel`, `PartidaModel`
Perfil y nivel	`PerfilController`, `PerfilService`, `NivelService`, `perfilView`, `public/js/perfil.js`
Foto de perfil	`FotoPerfilService`, `public/js/editor-foto.js`
Reportes	`ReporteService`, `ReporteModel`, modal en `juegoView`
Fondos, logo, emojis	`media/`, `MediaService`
Estilos	`public/css/zenit.css`
Deploy	`.github/workflows/deploy.yml`

Glosario rápido

PALABRA	QUÉ SIGNIFICA
POST / GET	Tipos de pedido. GET pide ver algo; POST manda datos (formularios, acciones del juego).
JSON	Formato de texto para intercambiar datos entre juego.js y el servidor.
fetch()	Función de JavaScript para hablar con el servidor sin recargar la página.
Sesión	Memoria del servidor asociada a tu navegador (cookie). Guarda quién sos y tu partida.

PALABRA	QUÉ SIGNIFICA
Consulta preparada	SQL con `?` en lugar de los datos: evita el SQL injection.
Hash	Transformación irreversible (contraseñas). Se puede comparar, no "descifrar".
Mustache	Motor de plantillas: completa los `{{huecos}}` del HTML con datos.
CDN	Servidor externo de donde se cargan librerías (Leaflet, qrcodejs).
Inyección de dependencias	Las clases reciben por constructor lo que necesitan (lo arma Configuration).